

Do Users Write More Insecure Code with AI Assistants?

Neil Perry, Megha Srivastava, Deepak Kumar, Dan Boneh
Stanford University · CCS 2023

arXiv: 2211.03622

Motivation: Why Study AI-Assisted Coding Security?

- AI code assistants (GitHub Copilot, OpenAI Codex) are rapidly adopted by millions of developers worldwide
- Prior lab studies showed AI tools can produce insecure code, but those used only synthetic prompts
- No large-scale study had examined real human interactions with AI assistants for security-sensitive tasks
- Key gap: how does natural, free-form developer usage affect security outcomes?
- Overconfidence risk: developers might trust AI output without verifying security properties
- This paper is the first comprehensive user study on AI assistant usage and code security across multiple languages

Three Core Research Questions

RQ1

Do users write more insecure code when given access to an AI programming assistant?

RQ2

Do users trust AI assistants to write secure code?

RQ3

How do users' language and behavior when interacting with an AI assistant affect the degree of security vulnerabilities in their code?

Study Design at a Glance

- 47 participants (after screening): 33 experiment group (with AI) + 14 control group (no AI)
- Recruited from 2 large US universities (undergrad & grad students) + industry professionals from 4 companies
- \$30 gift card compensation — study lasted up to 2 hours
- Participants solved 5 security-related programming tasks across Python, JavaScript, and C
- Tasks presented in randomized order to minimize fatigue and ordering effects
- All sessions screen-recorded (with consent) and code/interactions fully logged
- Experiment group had a secondary AI interface (OpenAI Codex davinci-002) to query freely
- IRB-approved; participants debriefed afterward about security analysis intent

Participant Demographics

Category	Details
Occupation	66% Undergraduate · 19% Graduate · 15% Professional
Age	87% aged 18–24 · 9% aged 25–34
Gender	66% Male · 28% Female
Programming Experience	62% under 5 years · 23% 5–10 years · 11% 10–15 years
Country of Origin	57% USA · 15% China · 13% India · 15% other
Group Split	33 Experiment (AI access) · 14 Control (no AI)

The 5 Security-Related Programming Tasks

Q1

Python

Encryption & Decryption — symmetric key AES

Q2

Python

Message Signing — ECDSA signing key

Q3

Python

Sandboxed Directory — restrict file access to /safedir

Q4

JavaScript

SQL Injection Prevention — parameterized DB inserts

Q5

C

Integer Overflow Safety — convert int to formatted string

The AI Assistant: OpenAI Codex (davinci-002)

- Model: OpenAI Codex codex-davinci-002 — the underlying engine of GitHub Copilot at the time
- Participants in the experiment group accessed a secondary UI to freely query the model
- No restrictions: participants could adjust temperature, rephrase prompts, chain outputs
- AI responses could be copied into the main code editor — or ignored entirely
- Model was pre-trained on large public code datasets (e.g., GitHub) — biasing toward common patterns
- Participants were NOT told the AI might produce insecure code — this was the key blind condition
- The control group had internet access but no AI assistant — reflecting real-world search-assisted coding

Section 2: Security Results Across All 5 Tasks

Top-Line Finding: AI Access → More Insecure Code

4/5

Tasks (of 5) where experiment group wrote more insecure code

67%

Experiment participants more insecure on Q1 Encryption ($p = 0.017$)

21%

Control group secure on Q2 Signing vs only 3% of AI group

AI users wrote significantly less secure code on 4 out of 5 tasks

Q1: Encryption & Decryption (Python)

✓ Control Group

- 79% of control group wrote correct solutions
- Control group used strong AES-GCM modes with authentication
- More likely to double-check library documentation
- No participants used trivial/substitution ciphers

✗ AI-Assisted Group

- 67% of experiment group correct — worse than control
- Significantly more likely to write insecure solutions ($p = 0.017$)
- AI often omitted authentication tags (EAX mode without returning tag)
- AI recommended trivial substitution ciphers in some cases ($p = 0.018$)
- Participants accepted AI output without verifying security properties

Q2: Message Signing with ECDSA (Python)

✓ Control Group

- 21% of control group wrote fully secure solutions
- Used deterministic ECDSA (RFC 6979) or safer libraries
- More careful library selection and verification

✗ AI-Assisted Group

- Only 3% of AI group wrote secure solutions ($p = 0.039$)
- AI chose libraries that explicitly flag insecurity in their own docs
- Unsafe randomness was the dominant error — directly from AI choice
- Participants trusted AI correctness without reading library warnings
- Example: Participant 1008 verified tests but not security — submitted insecure code

Q3 & Q4: Sandboxed Paths & SQL Injection

Key Patterns (Q3 & Q4)

- Q3 (Sandboxed Dir): No statistically significant difference overall
- However, AI-assisted users failed to block symlink traversal attacks
- AI solutions checked for '..' but missed `os.path.realpath()` patterns
- Q4 (SQL): AI group more likely to use string concatenation → SQL injection

Risks Observed

- Q4 Secure rate — Control: higher use of parameterized queries
- AI frequently suggested raw string formatting with user input in SQL
- Experiment participants rarely verified AI SQL output for injection safety
- Illustrates AI training data bias toward convenience over security

Q5: Integer Overflow in C — Mixed Results

- Task: Write a C function converting a signed integer to a formatted string (e.g. 7000000 → '7,000,000')
- This was the only task where AI users did NOT show clearly worse security outcomes
- However, participants took more than twice as long on Q5 (~31 min vs 11–14 min for others)
- C is not officially marketed by OpenAI for Codex — model showed weaker performance here
- AI suggestions in C often had buffer allocation issues but participants were more skeptical
- Result: AI assistance on lower-level or less-common languages may be less confidently adopted
- Highlights the importance of studying tasks across multiple programming languages

RQ2: Do Users Trust AI to Write Secure Code?

Higher

AI group participants who believed they wrote secure code (despite flaws)

Lower

Control participants more likely to express doubt about their solution's security



Key danger: overconfidence correlates with NOT verifying AI output

Participants with AI access were significantly more likely to rate their insecure solutions as secure — a 'false sense of security'

“

Participants with access to an AI assistant were more likely to believe they wrote secure code than those without access — highlighting the potential pitfalls of deploying such tools without appropriate guardrails.

— Perry et al., CCS 2023

Section 3: How Prompting Behavior Affects Security

Prompting Strategies That Improved Security

- Specifying task requirements explicitly in the prompt → more secure and correct outputs
- Providing function declarations or signatures for the AI to complete → better security outcomes
- Asking AI to focus on writing helper functions (not the entire solution) → reduces risk
- Increasing temperature and rephrasing prompts iteratively → improved security over time
- Participants who engaged more with prompt language and format produced fewer vulnerabilities
- Trusting AI less (i.e., being more skeptical) correlated with writing more secure code
- Key insight: Prompt engineering skill is now a security skill for developers

Prompting Patterns That Worsened Security

- Reusing AI output as the next prompt ('feedback loops') amplified or replicated vulnerabilities
- Accepting first AI response without any validation → most common path to insecure code
- Vague prompts lacking security context → AI produces functionally plausible but insecure code
- Not reading library documentation suggested by AI → hidden insecure defaults adopted
- Verifying only correctness (e.g., running test cases) but not security — a critical blind spot
- Example: Participant 1010 verified AES-EAX encrypted correctly, but missed missing auth tag
- Example: Participant 1008 ran tests, saw correct output, submitted — didn't check library warnings

Who Is Responsible for Security Mistakes?

AI-Originated

Mistake directly from AI output; user accepted without checking

User-Originated

Mistake introduced by user despite correct AI suggestion or independently

Internet-Originated

User copied insecure snippet from web resources (Stack Overflow, docs)

Statistical Modeling: Controlling for Confounders

- Used logistic regression to model security outcomes per task
- Controlled for: prior security coursework, years of programming experience, student status
- Removed collinear variables (education level & programming experience were correlated)
- Applied Benjamini-Hochberg correction for multiple comparisons
- Inter-rater reliability (Cohen's Kappa): 0.68–0.96 across questions — strong agreement
- Two independent raters coded all $47 \times 5 = 235$ solutions; disagreements resolved by majority vote
- Chi-squared tests (unequal variance) used for categorical comparisons between groups
- Welch's t-test used for continuous measures — robust to unequal sample sizes

Related Work: What Was Known Before

Prior Work

- Pearce et al. (2022): GitHub Copilot produced insecure code in ~40% of synthetic prompts
- No prior human user study with free-form interactions
- Codex evaluated mainly on HumanEval/MBPP for correctness — not security
- Google found 6% coding time reduction with AI — but didn't measure security

The Paper's Advances

- This paper: first large-scale user study with real developers on security tasks
- Multi-language study (Python, JS, C) — not just C
- Free parameter adjustment (temperature) and prompt rephrasing allowed
- Examined both security outcomes AND user trust/overconfidence
- Released anonymized data + UI for replication

Study Instrument & Reproducibility

- Custom desktop app built with React, Redux, and Electron (~4,000 lines of JSX)
- Two panes: (1) code editor + run output, (2) AI assistant query interface for experiment group
- All interactions automatically logged: queries, responses, accepted outputs, final code
- Participants accessed the study via a virtual machine on the researcher's computer
- Exit survey after each task: self-reported security confidence and experience
- All anonymized participant data, prompts, and the full UI are publicly released
- Goal: enable future replication studies and model comparisons as AI tools evolve
- IRB-approved at Stanford; all participants signed informed consent

Limitations & Threats to Validity

- Sample size: 47 participants is substantial for this type of study but limits statistical power
- 2:1 experiment-to-control ratio — deliberate but means fewer control observations
- Participant pool: predominantly young (87% aged 18–24), university-based — not fully representative
- Single AI model used (Codex davinci-002) — results may differ for newer models (GPT-4, Copilot X)
- Question 6 (XSS/DOM display) was found to be too vague — excluded from analysis
- Security classification required manual judgment — mitigated by dual coding and Kappa scoring
- Lab setting may not perfectly reflect production development pressure/context
- C tasks: Codex not officially supported for C, limiting direct comparison

Implications & Recommendations

For Developers

Treat AI output as a first draft. Always audit AI-generated code for security — especially cryptographic and input-handling code.

For Tool Builders

Integrate security linters and warnings directly into AI code assistants. Flag insecure library choices proactively.

For Educators

Teach prompt engineering as a security skill. Students need training in evaluating AI code, not just writing it.

For Researchers

Use the released dataset and UI to replicate and extend across newer models, more languages, and larger samples.

Key Takeaways

- AI assistants cause developers to write significantly more insecure code on 4 of 5 tested tasks

- AI users exhibit a 'false sense of security' — overconfident despite producing insecure code

- The danger is not that AI fails to code, but that it codes convincingly yet insecurely

- Better prompting (explicit specs, helper-function focus, skeptical iteration) reduces risk

- AI tools need built-in security guardrails — not just correctness feedback

- Released dataset + UI enables future research as AI coding tools continue to evolve